To build a complete application, you compose multiple middleware components into a pipeline, as shown in section 4.1. Each middleware component has access to the original request, as well as any changes made to the `HttpContext` by middleware earlier in the pipeline. When a response has been generated, each middleware component can inspect and/or modify the response as it passes back through the pipeline before it's sent to the user. This feature allows you to build complex application behaviors from small, focused components.

In the rest of this section, you'll see how to create a middleware pipeline by combining various middleware components. Using standard middleware components, you'll learn to create a holding page and to serve static files from a folder on disk. Finally, you'll take a look at a more complex pipeline such as you'd get in a minimal API application with multiple middleware, routing, and endpoints.

## *4.2.1 Simple pipeline scenario 1: A holding page*

For your first app in this chapter and your first middleware pipeline, you'll learn how to create an app consisting of a holding page. Adding a holding page can be useful occasionally when you're setting up your application to ensure that it's processing requests without errors.

**TIP** Remember that you can view the application code for this book in the GitHub repository at http://mng.bz/Y1qN.

In previous chapters, I mentioned that the ASP.NET Core framework is composed of many small individual libraries. You typically add a piece of middleware by referencing a package in your application's .csproj project file and configuring the middleware in Program.cs. Microsoft ships many standard middleware components with ASP.NET Core for you to choose